

NEW SYSTEMS

Making processing fail-safe

ROBERT FREIBURGHUSE, Stratus Computer, Inc.

Any number of modules, each configurable to be fully redundant, achieves 'continuous processing' for the Stratus/32

Buyers who must have a computer system that continues to function despite failures have not had much of a selection, but their choice has been broadened by the introduction of the Stratus/32 system from Stratus Computer, Inc., Natick, Mass. With 4M bytes of memory, dual 143M-byte disks, a 600-lpm printer and magnetic tape, a typical Stratus/32 sells for \$172,000, including COBOL and VOS software licenses. Based on COBOL benchmarks, the system's performance equals or surpasses that of several popular superminis.

The Stratus/32 multiprocessor, fault-tolerant system for commercial applications supports on-line transaction processing, batch processing, word processing and

interactive program development. It uses a combination of hardware and software that provides continuous processing of user programs during computer failure without checkpoint/restart programming at the user or system level. Central to the system's fail-safe operation are processing modules, each of which has redundant logic and communication paths, logic and CPU boards and main and disk memory. Twin components operate in parallel with each other; when one fails, its partner carries on.

Architectural overview

The Stratus/32 processing modules are connected via

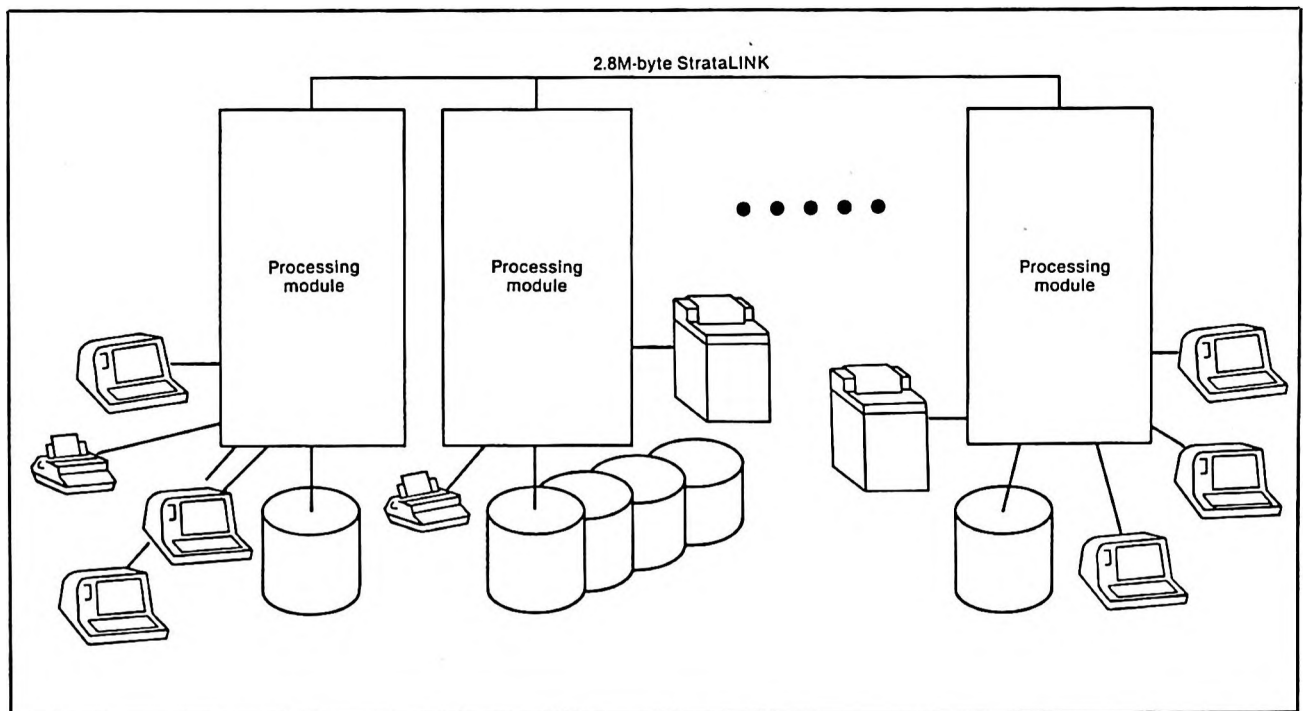


Fig. 1. A Stratus/32 system consists of as many as 32 processing modules connected by a high-speed coaxial link. The modules can be located anywhere within an office building or can be adjacent to each other.

Each processing module can be configured as fully redundant, partially redundant or non-redundant.

the StrataLINK high-speed coaxial link. Each processing module consists of memory, two Motorola Corp. 68000 CPUs, at least one disk and various peripheral controllers and devices (Fig. 1). Both 68000 CPUs are visible to the operating system, and each executes its own instruction stream using a shared memory.

Each processing module can be configured as fully redundant, partially redundant or non-redundant. The degree of a module's redundancy determines the module's resistance to hardware failure. A fully redundant module can withstand failure of essentially any component in the module without performance or data loss and without user program interruption. Multiple modules are used only to achieve greater system capacity; they never serve as backup for other modules.

Stratus's distributed virtual operating system (VOS) runs in each of the processing modules. All modules are equal and can operate independently, but through the use of transparent local networking software, VOS makes the entire set of processing modules appear as a single computer system to programs, programmers and application users.

Although each peripheral device is attached to a processing module, VOS makes all devices available to programs running in any module. Similarly, a program running in a module creates processes to run in the same module or in others. An interactive terminal user can create processes to execute commands or to run programs in any module. The users need not be aware

of the module they are using. Likewise, batch jobs can run anywhere in the system.

All VOS service requests have a uniform interface that is independent of the processing module on which the work will be performed. For example, a request to open a file has the same form and arguments regardless of where the file resides. VOS examines the file name, looks in a device table to determine the module that owns the device and performs the requested operation or makes a network request over the StrataLINK to the VOS running the owning module. The requesting program does not see the network request. Consequently, user programs are unaware of the location of files or devices and see the multiple-module network as a single virtual-computer system (Fig. 2).

Examining the hardware

A processing module includes one or more cabinets that contain a complete computer with a logic-board chassis, dual power supplies, peripheral devices and a terminal port. A single cabinet holds a fully redundant module consisting of two 143M-byte disk drives, a magnetic tape, 16M bytes of memory, redundant CPU boards and a set of redundant peripheral controllers (Fig. 3).

A high-speed bus with a 125-nsec. cycle time is central to processing-module organization. The bus—virtually two buses operating in parallel—has two sets of data and control-logic paths. The data path on each bus is 32 bits wide, and data can be put on the bus every bus cycle. This results in a potential bus rate of 32M bytes per sec., although processor/memory boards now run at 16M bytes per sec. By comparison a VAX/11-780 bus runs at 13M bytes per sec.

Each logic board that can be attached to the bus can

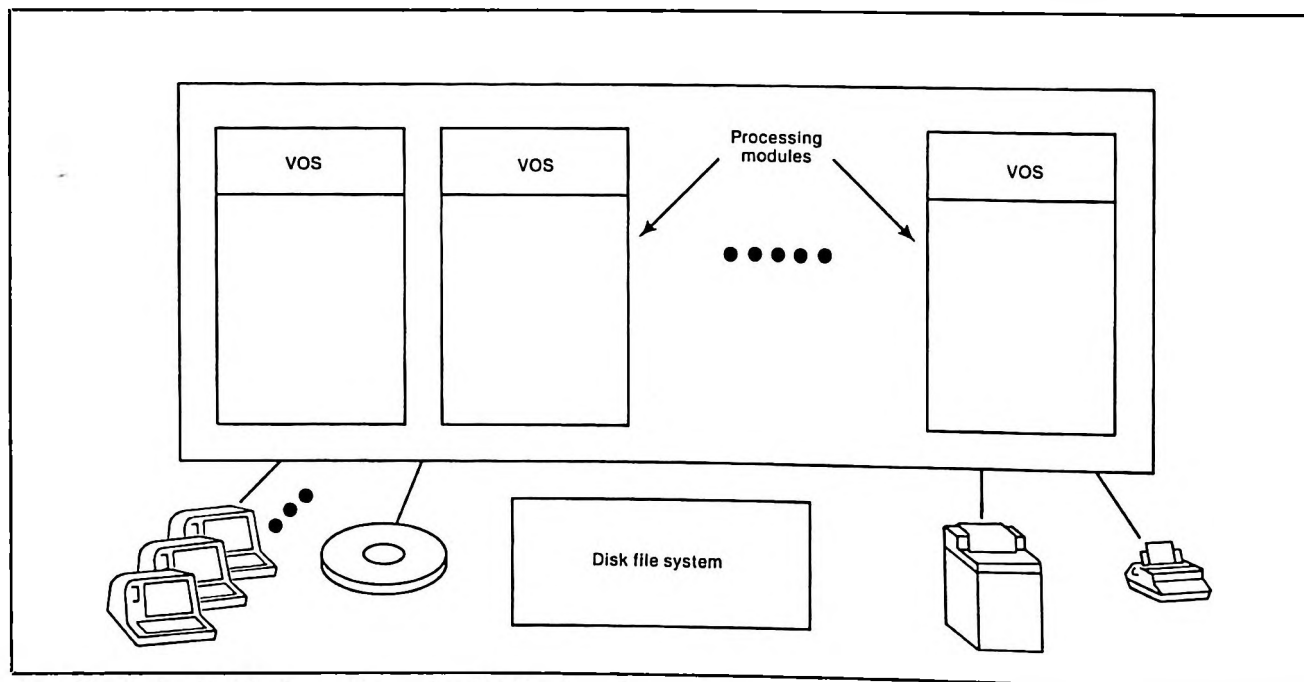


Fig. 2. The Stratus/32 virtual operating system (VOS) makes all processing modules appear as part of a single virtual system in which all devices, files and system resources are accessible as if it were a single computer.

The CPU board contains two complete sets of logic and is self-checking.

However, the operating system has sufficient software error detection to run dual StrataLINKs as separate parallel links until one fails. Failure of a link is detected, and data are retransmitted over the other link without affecting users of the link. A link failure could cause some performance loss because a dual link has a 2.8M-byte-per-sec. transmission rate, while a single link has a 1.4M-byte-per-sec. rate. But because most links do not operate even close to their maximum rates, it is unlikely that an application would notice the performance difference.

A failure scenario

When a logic board or an attached peripheral device fails, it puts itself off-line, lights a red light on the board and transmits an interrupt to the operating system. Maintenance software in the system then tests the failed board to determine if the failure was transient or

hard. In either case, the failure is noted in a hardware-failure log file, and selected terminals are notified of the failure. If the board passes the maintenance-software check, it is resynchronized with its redundant partner and put back on-line, and its red light is turned off. If the board fails the software check, it remains off-line, and a red light on the system control panel is lit.

A failed board can be replaced in a running system by a nontechnical person without special tools and without affecting any user's program. VOS dynamically reconfigures itself when a board is added or removed from the system.

Operating-system software

Each processing module contains two software-visible 68000 processors and a software-visible Z80 μ p on each peripheral controller. The VOS off-loads detailed I/O processing to the Z80s and uses one of the 68000s to respond to interrupts and to execute the word processor. The other 68000 executes user code and non-interrupt-driven system code. For example, a user program that calls VOS to perform disk I/O has the user 68000 enter VOS and set up the disk-I/O command chain. The user 68000 is then rescheduled to execute another user's program. The disk-I/O operations are executed by the Z80 on the disk controller, and when the disk operation completes, the "executive" 68000 responds to the interrupt and posts a notification to the scheduler that this I/O event is complete. Depending on the setting of the scheduler's parameters, the user 68000 can be rescheduled to execute the user's program then or at the next scheduling interval.

A "process" is a virtual CPU and a 16M-byte address space in which a sequence of user programs can be executed. A process is created when a user logs onto the system, and is terminated when a user logs off. A process can create additional processes that operate independently of the creating process. VOS uses system processes to monitor terminals for log-in requests, to run the spooler and to perform requests made by other processing modules.

Each process has an address space consisting of 4M bytes for VOS and 12M bytes for a user program (Fig. 6). A program consists of any number of separately compiled subroutines, which can be written in different source languages.

The 16M-byte virtual address space is divided into 4096-byte pages that are mapped into physical memory pages by an address-translation map on the CPU board. The translation occurs within the 125-nsec. clock cycle and does not slow the CPU. The user's program and most of the programs in VOS see only the virtual address space and are unaware of the translation.

The hardware address-translation map informs the scheduling and paging algorithms of which physical pages were recently used and which virtual pages of each process were recently used by that process. Thus, it is possible to determine quickly the real working set

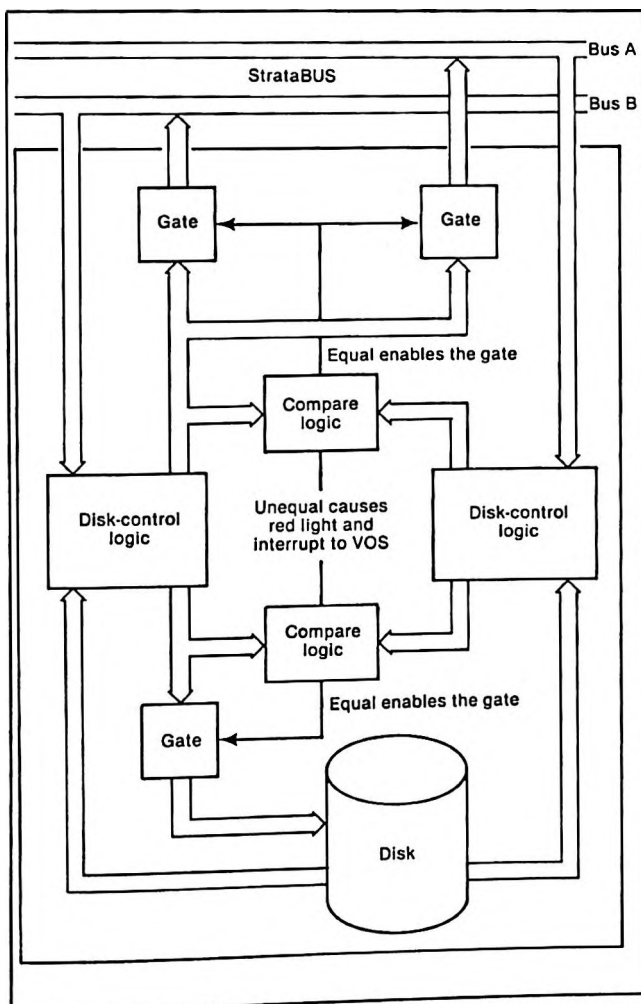


Fig. 4. Self-checking disk controller. Two sets of logic operate in parallel with each other. If their results differ, a warning light goes on, and no data are sent until the situation is corrected.

When data are written to a given memory location, both memory controllers respond and write the data into their memory.

of each process and which physical page of memory to replace when a new page is required.

Sharing and protection.

Each page of a process's address space has an associated access code that allows execution, read/write or read-only. The processor has supervisor and user execution states, and, for each state, it has potentially separate access codes for each page of the address space. These access codes are stored in the address-translation map used to translate virtual to physical addresses, and are enforced for every memory reference in both user and supervisor states.

All users share the physical pages of VOS and appear in the address space of each process. Likewise, user programs are shared if executed by more than one process. Sharing user programs and system commands and utilities requires no action on the part of a user and occurs with no visible effect to the user. All code produced by Stratus compilers is pure and reentrant.

A user's program is protected from destruction because each page of program instructions is given an execute-only access code. A user's data are protected from other users because they exist only in the address space of the user. The data are also protected from execution to aid debugging of users' programs.

Inter-process communication facilities of VOS allow a multi-process application to be developed on a single-module configuration and run later without modification on a distributed configuration consisting of multiple processing modules, or even on multiple Stratus systems connected by StrataNET. Consequently, the mechanisms used to start, stop or synchronize processes do not use shared memory; they use the file system as a high-level shared memory.

Every file in the file system has an associated lock and an associated event. In addition, each record in a file has an associated lock. Locks synchronize file access between two or more processes and can be set or reset to indicate reading or updating of the file or records.

Simple notification and waiting for events between processes are performed using event counts. An event count is a large integer that is incremented each time that it is notified by a process.

Most inter-process communication is related to files and consists of notifying processes that data are available to be processed or of waiting for data to arrive for processing. The use of events and locks associated with files is consequently natural and efficient.

User-visible software

The Stratus/32 was introduced with a host of native-mode software, including:

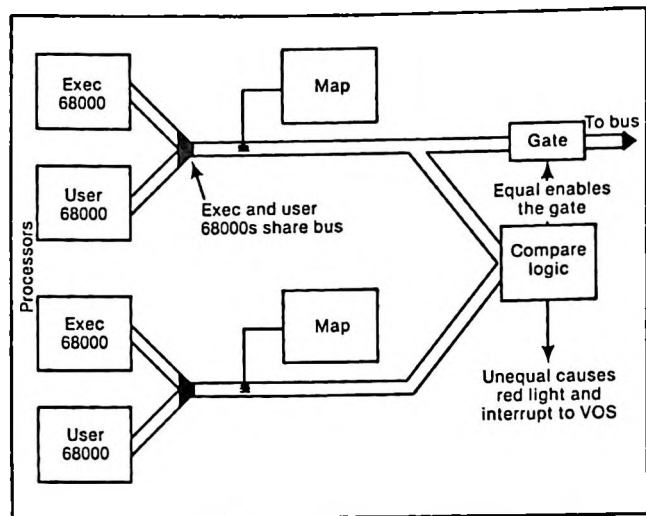


Fig. 5. A single Stratus/32 CPU board contains four Motorola 68000 processors that provide two software-visible processors. The board is fully self-checking and contains redundant virtual/physical address-translation maps.

- **CRT menu-oriented commands.** The VOS command interpreter accepts all commands in two forms: a conventional line-oriented form suitable for use on hard-copy terminals or from within command language programs called command macros, and a CRT menu form that displays all command options and their default values. The CRT menu form is invoked by typing the name of the command followed by a function key. A command reference manual is unnecessary.

- **Five industry-standard languages.** These include COBOL 74, FORTRAN 78, PL/I-G, Pascal and BASIC. All languages are supported by optimizing compilers that share an optimizer and a code generator that produces highly optimized relocatable binary code. Subroutines written in all languages can be combined into a program, and all languages can call all VOS service subroutines. All compiled code is pure and reentrant.

- **Program debugging.** Program development in any high-level language can be done entirely within that language without the programmer's knowing the instruction set, data formats or register arrangement of the 68000 processor. For all languages, the debugger can set breakpoints on statements, display variables, make assignments to variables, execute calls and functions, execute gotos, step through the program and execute conditional breakpoints. The debugger can be entered to start execution of a program, when a program fails or to examine a snapshot of a program that has failed.

- **Data security.** Each file has an access-control list consisting of pairs of user IDs and associated access rights (execute, read or read/write). Access rights can be specified by a user or groups of users and are enforced by the VOS I/O system. Access-control lists provide file security without embedding passwords into programs. The lists operate on the basis of people or groups who access the file; consequently, they are easy to administer and use. Security is provided regardless

The bus—virtually two buses operating in parallel—has two sets of data and control-logic paths.

detect its own failure and shut itself down. It can also run with a redundant board that continues to operate in the event of its partner's failure. Neither logic board is primary, and neither is aware of the other. The pair of boards appears collectively to other system components as a fail-safe entity.

The self-checking technique used by each type of board differs slightly, but generally involves the use of two sets of logic on a board. Each set performs every operation in parallel with the other. When data are to be sent to the bus or to a device, the results produced by the two sets of logic are compared. If identical, the result is sent onto the bus or to the device. Dissimilar results indicate a board failure, and no data are sent. In this case, a red LED on the board is lit, and an interrupt signal is sent on the bus. Until the board is tested and logically reconnected by maintenance software, it remains off-line. The board's redundant partner continues to operate, and no other component of the system is aware of the failure (Fig. 4).

The CPU board contains two complete sets of logic

and is self-checking. Four Motorola 68000 processors provide each board with two processors visible to the operating system (Fig. 5). A redundant partner CPU board ensures continuous processing in the event of a failure of a CPU board. At a component price of approximately \$100 for each 68000, this is a cost-effective solution to continuous processing that was not practical until the availability of VLSI technology.

Redundancy is achieved by using a pair of logic boards for each logical entity in the system. Each board is attached to both halves of the bus, and both boards operate in parallel, performing identical operations. The output of both boards is placed on the bus at the same instant and is guaranteed to be identical.

Memory is duplicated in a redundant system so that N megabytes of program-visible memory is implemented using 2N bytes of physical memory with N megabytes attached to each of two memory controllers. When data are written to a given memory location, both memory controllers respond and write the data into their memory. When data are read from memory, both controllers respond and read from their memory.

The controllers and the memory are synchronized and appear to the rest of the system as a single logical entity. Memory subsystems are not paired with CPUs, bus halves or other system components. Memory is implemented from 64K RAMs and is packaged on 1M- or 2M-byte boards. It has a 375-nsec. read-cycle time and is four-way interleaved. A typical processing module has 4M or 8M bytes of memory. In packaged configurations, Stratus sells memory for \$5000 per megabyte. A 2M-byte array board lists for \$20,000.

The memory system can be dynamically reconfigured to be redundant or non-redundant. This allows a module to use all available memory when full redundancy is not needed. Reconfiguration can occur on-line without affecting running programs.

Disks cannot run completely synchronized with each other; they require help from the operating system to provide continuous processing. Each disk can be configured to have a duplicate. The mirror disk is attached to a separate controller to protect from controller failure. When a program writes a record to a redundant disk, the operating system writes records to the disk and to its mirror. When a program reads from the disk, the operating system reads from the disk that is not busy or whose heads are best positioned to read the record. If a read error occurs, the record is read from the other disk.

An error-correction code stored with each disk record detects media failures during a read. A read error from a redundant disk results in reading the same record from its partner. Non-redundant disks are vulnerable to total disk failure, but are protected from media failures by VOS, which verifies each write. A record that cannot be verified is rewritten to another disk block, and the bad block is removed from the available disk space.

StrataLINKs, like disks, cannot run synchronized.

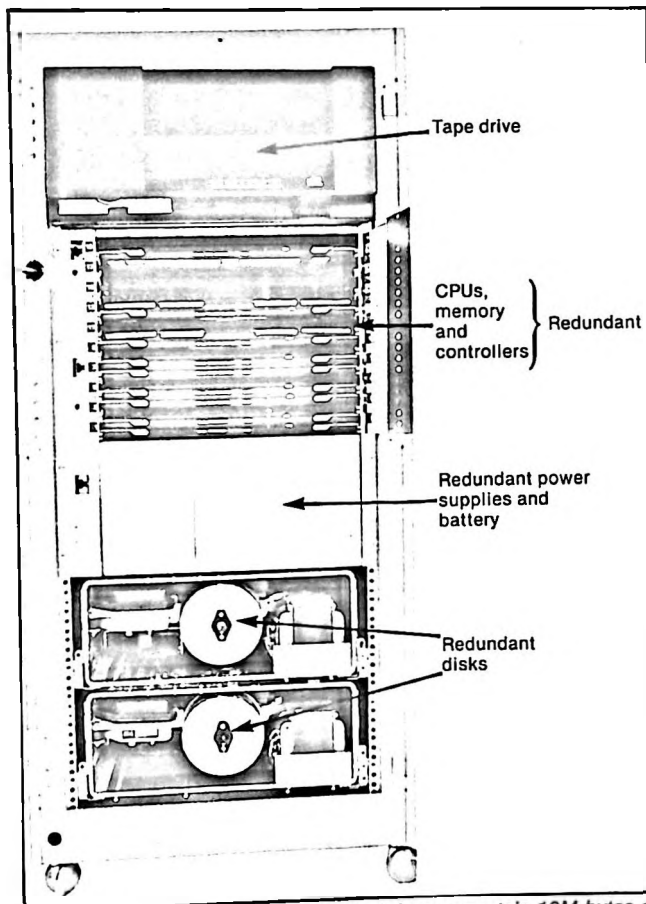


Fig. 3. A Stratus/32 processing module can contain 16M bytes of memory, a full set of redundant controllers, two 143M-byte disks, a tape drive and two software-visible CPUs. (Additional disks and tapes are held in adjacent cabinets.)

All languages are supported by optimizing compilers that share an optimizer and a code generator that produces highly optimized relocatable binary code.

of what programs or system commands are used to access files.

• **Networking.** StratanET permits two or more Stratus systems to run as if they were one system. Just as users of individual processing modules of a system have access to their entire system, users of a networked system have access to the entire network without any network-oriented requests or commands. Normal file operations and inter-process communication operate transparently to the user's program.

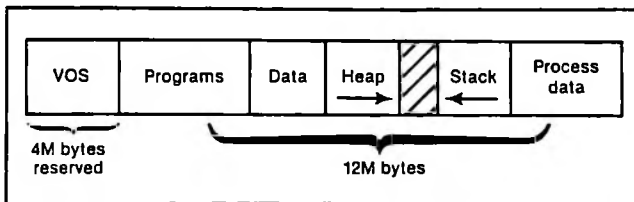


Fig. 6. Stratus/32 virtual address space provides as much as 12M bytes of program and data space per user program. The stack is the interrupt directory; the heap is handy for temporary data that must be stored randomly. Both expand and contract as needed. User programs and system code are shared without user involvement, and no segment limits are imposed on programs or data.

• **Transaction processing.** VOS supports the development of transaction-oriented applications by providing CRT forms-design utility and forms I/O statements in all high-level programming languages, multi-terminal transaction control in all high-level programming languages, individual record and file locking, multi-key indexed file access, queued file access and file-system operations to ensure the integrity of transactions.

Data representing a transaction can be processed directly by a user-written transaction-control program, or data can be written into a queued file to be processed by other programs, called transaction servers, which operate asynchronously with respect to the transaction-control program. Transaction servers can run in any processing module, not just modules running the transaction-control program. Likewise, transaction-control programs can run in several modules. The number and distribution of transaction servers can be dynamically altered in response to changes in the transaction rate. ■

Robert Freiburghouse is co-founder and vice president of software engineering, at Stratus Computer, Inc., Natick, Mass.

Reprinted from **MINI-MICRO SYSTEMS** May, 1982
© 1982 CAHNERS PUBLISHING COMPANY

STRATUS COMPUTER, INC
17-19 STRATHMORE ROAD
NATICK, MA 01780
(617) 853-1466

Stratus